

S D-PA R O

S e c u r i t y

&

R e s p o n s e

- Detects threats and prioritizes alerts
- Automates incident response
- Software Design & Architecture · Riphah International University

Muhammad Jaffar Nawaz · Mustafa Tahir · Ibad ur Rehman

Instructor: Sir Uzair Rasheed



The

Security teams face too many alerts and too little time.

- **Alert overload:** High volumes hide important signals.
- **Slow response:** Delays increase breach impact.
- **Inconsistency:** Manual triage leads to uneven decisions.

The result: alert fatigue, missed threats, and reduced confidence in response.



C u r r e n t

Cyber threats are rising in volume and sophistication, putting SOC's under heavy pressure.

3 %

R i s e

Threats continue to surge.

1M +

D a i l y

Manual triage is overwhelmed.

3

T a l e n t

Skilled defenders are scarce.

2

A v g .

Breaches can go unnoticed for months.

Automation is now essential for keeping pace.



M a r k e t

Automation is becoming essential in cybersecurity.



S t a n d a r d i z i n g A u t o m a t i o n

Automated detection and response is now expected.



C o m p e t i t i v e

Faster response and lower operational costs.



M a r k e t

Threats, compliance, and talent shortages.

W h a t -P R O ?



Security Detection and Automated Response Platform

- Reduces alert fatigue
- Speeds up incident handling
- Improves response consistency



P r o j e c t

SDA-PRO focuses on three core goals:

S t r e a m l i n e

Cut false positives and alert fatigue.

S p e e d

Automate investigation and remediation.

I m p r o v e

Use repeatable, context-rich workflows.

Scope covers the full alert lifecycle: ingestion, enrichment, correlation, and orchestration.

Layered

5 layers separate UI, orchestration, business logic, data, and external integrations.

Combines **MVC** for presentation/control with **SOA-style** service boundaries.

Presentation

Java Swing dashboard for analysts.

Displays alerts, incidents, and actions; no business rules.

Controller

AlertController, IncidentController, and IntegrationController bridge UI and logic.

They validate input and coordinate service calls.

Service

Holds core workflows: correlation, prioritization, and response sequencing.

Centralizes business rules and keeps logic reusable.

Integration

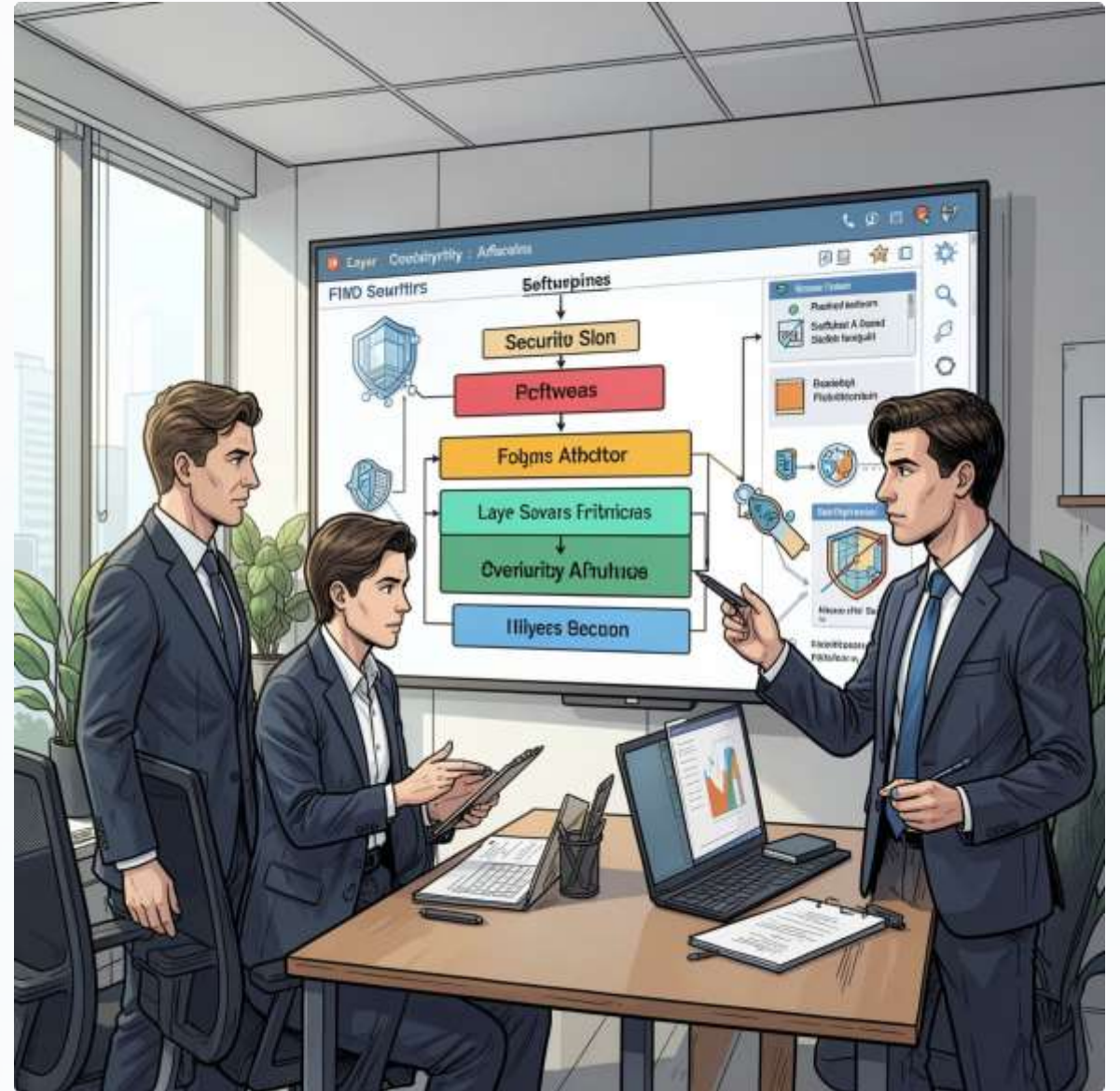
Connects to VirusTotal, MISP, Slack, and PagerDuty.

Isolates API details and external failure handling.

Repository

Abstracts persistence for alerts and incidents.

Keeps storage details out of the rest of the application.



C o m p o n e n t

SDA-PRO moves security events through a clear, layered pipeline from ingestion to response.

Untitled

[View more](#)

This flow shows how raw alerts become actionable intelligence and automated response.

I n t e g r a t i o n

Ingests external alerts and triggers actions.

R e p o s i t o r y

Stores alerts and incidents for analysis.

S e r v i c e

Enriches, correlates, and prioritizes events.

C o n t r o l l e r

Routes UI requests to the right services.

P r e s e n t a t i o n

Gives analysts a GUI to monitor and investigate.

A l e r t

SDA-PRO turns raw security events into standardized, actionable alerts.



I n g e s t i o n

Collect alerts from SIEMs, EDRs, and cloud logs.



P a r s i n g

Map data to a common schema.



E n r i c h m e n t

Add threat, asset, and user context.



F i l t e r i n g

Reduce noise and remove duplicates.



P r i o r i t i z a t i o n

Assign severity and risk score.



S t o r a g e

Store alerts for search and audit.

Consistent processing improves detection, correlation, and response.

U M L

Five UML diagrams show the system's core security workflow.



A l e r t

Normalizes raw alerts into a common format.



I n c i d e n t

Tracks incidents from New to Closed.



R e s p o n s e

Selects and executes remediation playbooks.



E v e n t

Broadcasts updates to dashboards, audits, and alerts.

Together, they separate concerns while keeping the response workflow connected end to end.



1 Design

SDA-PRO uses design patterns to keep the architecture modular, testable, and easy to extend. They reduce coupling across ingestion, normalization, orchestration, response, and reporting. Below is a concise breakdown of each pattern and its role in the system.



Factory

Creates alert and normalizer objects without hard-coding classes.



Abstract

Builds consistent notification components for Email, Slack, and PagerDuty.



Singleton

Provides one shared instance for configuration and event publishing.



Strategy

Switches response behavior by incident severity and context.



Proxy

Controls access to sensitive threat-intel lookups.





State

Models the 7-step incident lifecycle with explicit transitions.



Adapter

Normalizes external APIs like VirusTotal, MISP, Slack, and PagerDuty.



Observer

Broadcasts updates to dashboard, audit, and metrics subscribers.



Chain

Processes alerts through 5 enrichment handlers in sequence.



Decorator

Adds audit, approval, metrics, and rollback behavior around core logic.



Composite

Treats alert groups and single alerts through one interface.



P a t t e r n

The 12 design patterns work together to keep SDA-PRO modular, extensible, and easy to maintain.

C r e a t i o n a l

- **Factory Method** and **Abstract Factory** handle object creation
- **Singleton** centralizes config and event publishing

S t r u c t u r a l

- **Composite** groups alerts into campaigns
- **Adapter, Proxy, Decorator, and Facade** simplify integration and access

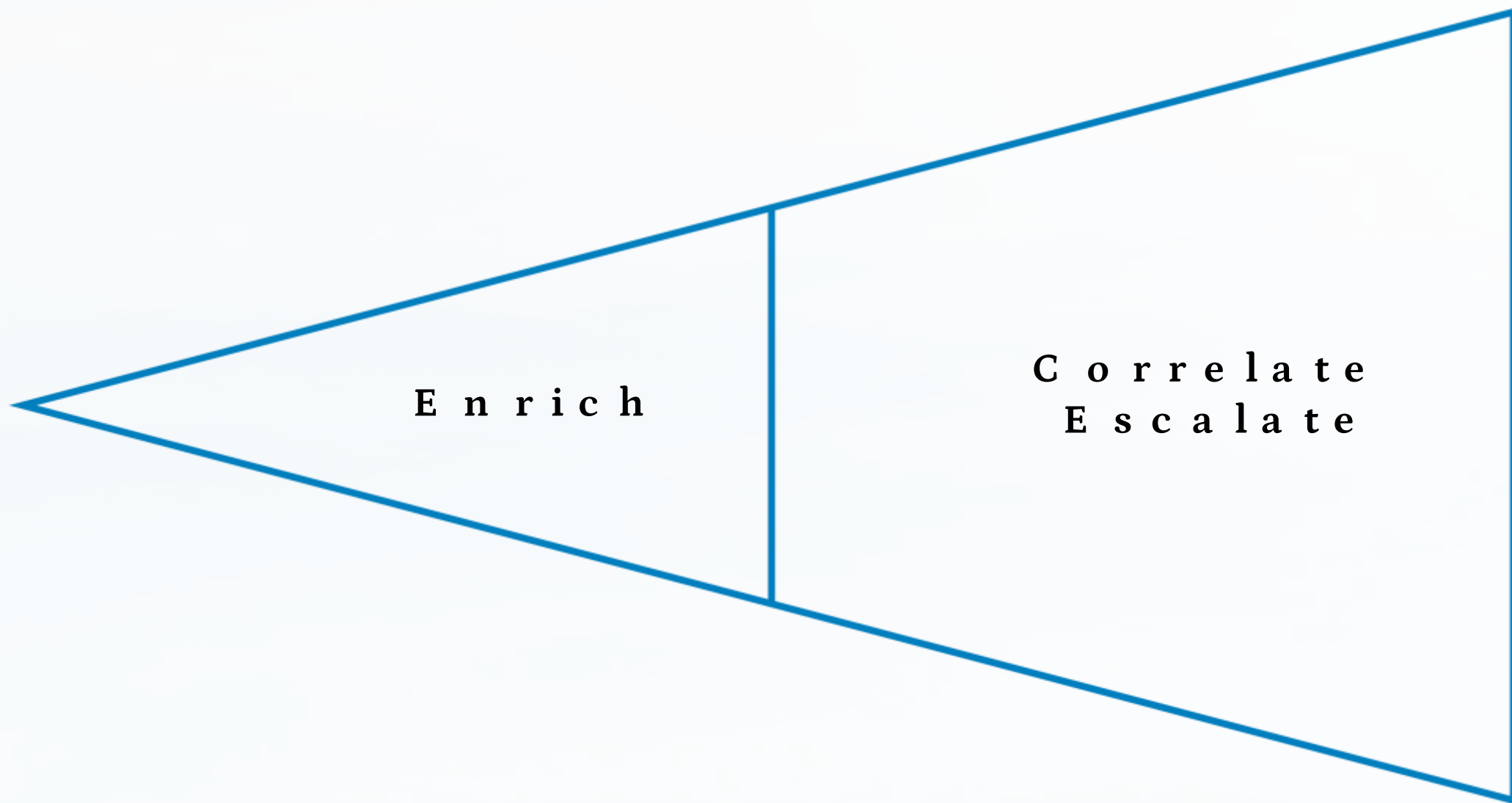
B e h a v i o r a l

- **Strategy** selects response playbooks
- **State, Observer, and Chain of Responsibility** manage lifecycle, notifications, and processing flow

Together, they connect creation, orchestration, response, and visibility into one cohesive flow.

E n r i c h m e n t

- Transforms raw alerts into actionable intelligence
- Automates context gathering and threat linking
- Reduces response time by prioritizing real threats
- Gives analysts a clearer view of attack campaigns



Incident — State

State Pattern guides incidents through 7 lifecycle stages.

🕒 Transitions happen through **nextState()**, keeping the Incident class simple.

New

Incident created and prioritized.

Containment

Limit damage and stop spread.

Recovery

Restore services and monitor closely.

Closed

Incident resolved and archived.

Triage

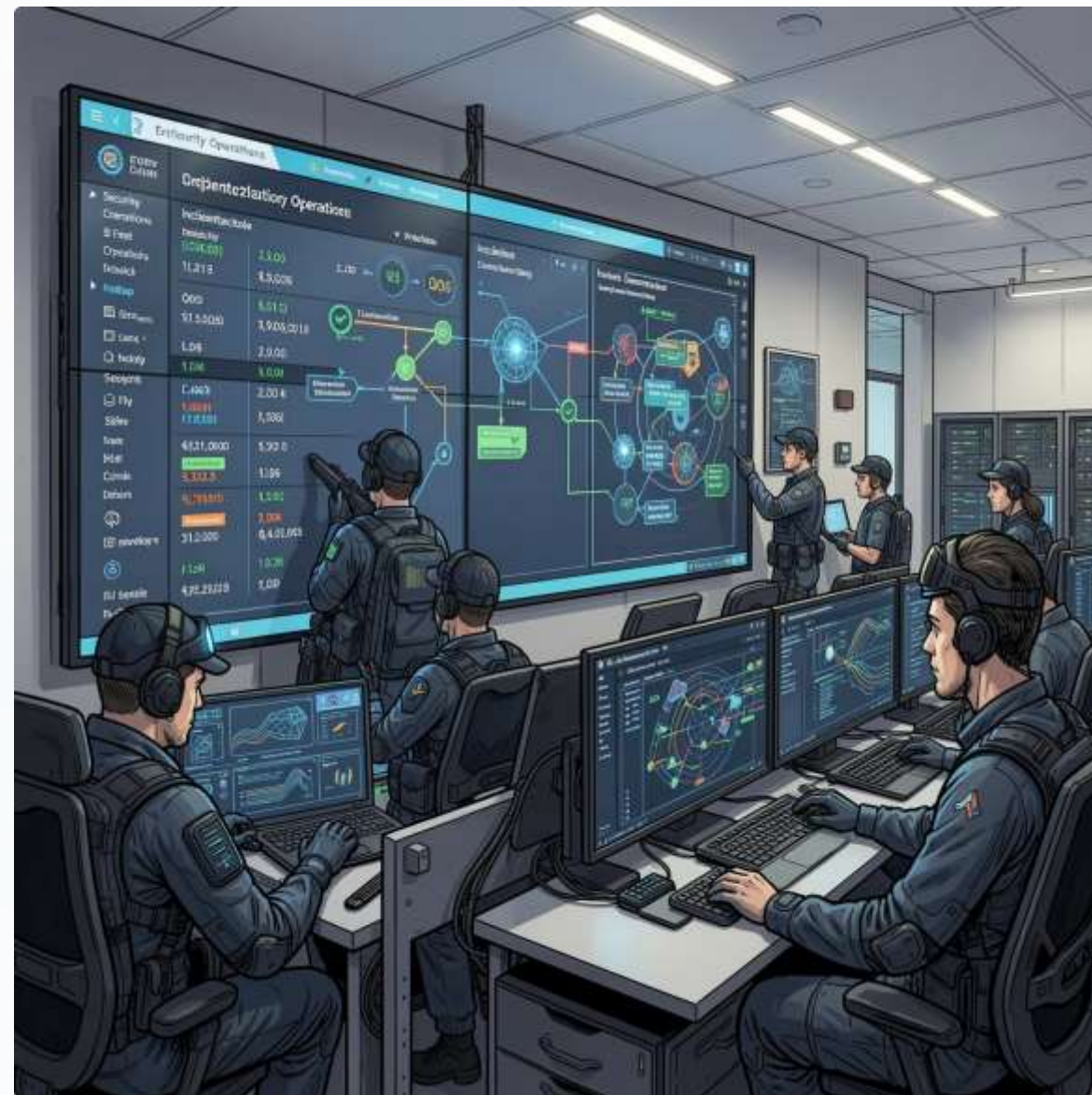
Assess scope, impact, and urgency.

Eradication

Remove root cause and threats.

Post Incident

Capture lessons learned and improvements.



R e s p o n s e

SDA-PRO adapts incident response based on threat severity, confidence, and risk tolerance.



A g g r e s s i v e

Critical threats; immediate containment and eradication.



B a l a n c e d

Suspicious activity; automated response with human review.



C o n s e r v a t i v e

Low-confidence signals; monitor, log, and gather intelligence.

Decision rules combine threat intelligence, asset criticality, and incident history.



A u t o m a t e d

SDA-PRO turns response strategies into executable workflows for fast, consistent incident handling.



R e s p o n s e

Pre-approved action sequences for specific incident types.



A u t o m a t e d

Blocking, isolation, disabling accounts, and quarantining data.



O r c h e s t r a t i o n

Coordinates execution, dependencies, and error handling.

Security teams respond at machine speed with human oversight.



G U I

I n t e g r a t i o n s

Java Swing: lightweight, cross-platform, no external dependencies.

D a s h b o a r d

Live incidents, alerts, response actions, and system metrics.

I n t e g r a t i o n s

VirusTotal, MISP, Slack, and PagerDuty via the Adapter Pattern.

E v e n t

EventBusPublisher keeps the UI, logs, and metrics in sync.



Chosen for mature desktop performance and easy offline packaging.

Integration

Integrates with external services for threat intelligence, notifications, and incident response.

VirusTotal

File reputation and scan results.

MISP

Threat sharing and indicator correlation.

Slack

Team alerts and operational visibility.

PagerDuty

Escalates urgent incidents to responders.

All integrations use the **Adapter Pattern** for a consistent internal interface.

Audit Compliance

SDA-PRO tracks system activity for security, compliance, and investigations.



C o m p r e h e n s i v e

Records logins, changes, and configuration updates.



C o m p l i a n c e

Supports GDPR, HIPAA, and ISO 27001 audits.



F o r e n s i c

Provides timestamps, user IDs, and activity details.



T e s t i n g

J U n i t

8 test classes verify core patterns and workflows:

- **AlertFactoryTest** — alert creation and type mapping
- **CompositeTest** — incident grouping and traversal
- **ConfigManagerTest** — config loading and overrides
- **DecoratorTest** — alert enrichment behavior
- **FacadeTest** — simplified subsystem access
- **IncidentStateTest** — lifecycle transitions
- **ObserverTest** — event notifications
- **ResponseStrategyTest** — strategy selection

01

A D-OR

Layered Architecture

03

A D-OR

Event Bus

M a v e n

- `mvn clean` — fresh build
- `mvn test` — run JUnit suite
- `mvn clean package` — build artifact

Repeatable Maven flow supports reliable releases.

A r c h i t e c t u r e

6 ADRs document the key technical choices.

They preserve traceability and explain tradeoffs.

02

A D-OR

Pattern Selection

04

A D-OR —O

GUI, Integrations, Persistence

P e r f o r m a n c e

Real-time monitoring keeps the platform secure and stable.

S y s t e m

- Latency
- Throughput
- CPU and memory usage

I n c i d e n t

- MTTD
- MTTR
- Faster resolution workflows

O p e r a t i o n a l D a s h b o a r d s

- Real-time visibility
- Threat trend tracking
- Response impact review

These dashboards help teams spot issues quickly and respond effectively.

Key

Our architecture prioritized modularity, extensibility, and clarity.

Design

Used 12 patterns to keep the codebase modular and maintainable.

State-Derived

State Pattern enforced clear incident progression and prevented invalid actions.

Decoupled

Observer-based event bus enabled real-time updates across components.

Integration

Adapter Pattern standardized connections to external services.

Desktop

Java Swing provided a lightweight, reliable, platform-independent interface.

Quality

JUnit tests and ADRs supported validation and long-term maintainability.

Challenges

Key technical challenges were resolved with patterns, clear architecture, and event-driven design.

Architecture

12 patterns required strict layering and separation.

External

Adapters unified APIs like VirusTotal, MISP, Slack, and PagerDuty.

Incident

State Pattern kept transitions valid and predictable.

Re-aim

Observer bus delivered fast, decoupled system updates.

R e s u l t s

1

D e s i g n

Factory, Strategy, State, Observer

5

A r c h i t e c t u r e

Presentation to Integration

8

J U n i t

All passed

4

I n t e g r a t i o n s

VirusTotal, MISP, Slack, PagerDuty

- Clean layered architecture improved maintainability and extensibility.
- Design patterns reduced coupling and made the system easier to test.
- Successful Java application with external integrations and full automated coverage.
- Future work: richer analytics, dashboards, and automation.

MVC

SOA

JAVA

CYBERSECURITY

DESIGN PATTERNS

MAINTAINABILITY

EXTENSIBILITY



Future Scalability

Built to evolve, scale, and support new capabilities over time.

Feature



- AI/ML threat intelligence
- Advanced analytics dashboards
- More SIEM/SOAR integrations

Scalability



- Cloud-native microservices
- Database sharding and replication
- Elastic incident processing

Architectural



- Modular, pattern-based design
- Easy to add new modules
- API-first integration support

